



Docker is an open source project, which packages applications and their dependencies in a virtual container that can run on any Linux server. Docker has immense possibilities as it facilitates the running of several OSs on the same server.

Technology is changing faster than styles in the fashion world, and there are many new entrants specific to the open source, cloud, virtualisation and DevOps technologies. Docker is one of them. The aim of this article is to give you a clear idea of Docker, its architecture and its functions, before getting started with it.

Docker is a new open source tool based on Linux container technology (LXC), designed to change how you think about workload/application deployments. It helps you to easily create light-weight, self-sufficient, portable application containers that can be shared, modified and easily deployed to different infrastructures such as cloud/compute servers or bare metal servers. The idea is to provide a comprehensive abstraction layer that allows developers to 'containerise' or 'package' any application and have it run on any infrastructure.

Docker is based on container virtualisation and it is not new. There is no better tool than Docker to help manage kernel level technologies such as LXC, cgroups and a copy-on-write filesystem. It helps us manage the complicated kernel layer technologies through tools and APIs.

What is LXC (Linux Container)?

I will not delve too deeply into what LXC is and how it works, but will just describe some major components.

LXC is an OS level virtualisation method for running multiple isolated Linux operating systems or containers on

a single host. LXC does this by using kernel level name space, which helps to isolate containers from the host. Now questions might arise about security. If I am logged in to my container as the root user, I can hack my base OS; so is it not secured? This is not the case because the user name space separates the users of the containers and the host, ensuring that the container root user does not have the root privilege to log in to the host OS. Likewise, there are the process name space and the network name space, which ensure that the display and management of the processes run in the container but not on the host and the network container, which has its own network device and IP addresses.

Cgroups

Cgroups, also known as control groups, help to implement resource accounting and limiting. They help to limit resource utilisation or consumption by a container such as memory, the CPU and disk I/O, and also provide metrics around resource consumption on various processes within the container.

Copy-on-write filesystem

Docker leverages a copy-on-write filesystem (currently AUFS, but other filesystems are being investigated). This allows Docker to spawn containers (to put it simply—instead of having to make full copies, it basically uses 'pointers' back to existing files).